



南開大學
Nankai University

计算机学院
语音信息处理实验报告

实验六：基于 AI Coding 的语音识别

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 6 月 27 日

目录

1 实验任务	2
2 基础任务	2
2.1 实验目标	2
2.2 实验数据与环境	3
2.3 实验操作与代码实现	3
2.4 实验结果	5
2.5 折线图生成脚本	7
2.6 对本项目的理解	7
3 进阶任务	8
3.1 进阶任务目标	8
3.2 进阶任务做了什么	8
3.3 进阶模型配置	9
3.4 训练结果	10
3.5 测试集结果	11
3.6 与基础模型的结果对比	11
3.7 预测样例对比	12
3.8 为什么 Conformer 有效果	13
4 关于使用 AI 的一些问题	13
4.1 AI Coding 模式体验	13
4.2 如何定位和修正 AI 代码问题	14
4.3 我和 AI 的长期关系与核心壁垒	14

1 实验任务

- 基础任务：基于 AI Coding 或者 Vibe Coding 完成训练代码，推理代码以及评测代码。
 - 经个人验证，豆包可以很好地完成任务，最后的 WER 在 30% 以内。
 - 由于模型参数量比较小且任务简单，上述是在训练 10 个 epoch，找验证集上最优的 epoch 来进行测试的结果。（CPU 即可，不希望大家花钱租 GPU，本次实验的目的仅仅就是希望大家明白语音识别怎么做的，并且学会使用 AI 完成任务）。
 - 不需要去卷最后的性能。正常训练就好。如果实在训不出来，需要分析失败原因。
- 进阶任务：和 AI 讨论，利用 AI 做出改进。改进不好也没有关系，需要分析原因。包括但不限于：替换 encoder 框架，使用一些预训练的 encoder 训 CTC，等等。
- 本实验需要提交一个实验报告，内容要求
 - 记录实验结果，并简单介绍一下你对本项目的理解
 - 如果完成进阶任务，需要描述进阶任务做了什么，为什么有效果/没效果
- 回答关于使用 AI 的一些问题：
 - 本次实验是一个完整的 ASR 项目，而非单文件单函数的简单任务。那么你在开发过程中，主要使用的是「单模块 / 单函数逐段补全的传统 AI Coding 模式」，还是「基于全项目上下文、支持多文件联动修改的 Vibe Coding 模式」？请你结合本次使用体验，说说你的感受。
 - 在使用 AI 的过程中，AI 生成的代码一定会出现不符合预期、甚至完全错误的情况。那么你是如何定位到问题根源的？又是如何修正代码解决问题的？
 - 你认为你和 AI 会形成怎样的长期共生关系？AI 会成为你的什么角色？你会如何构建自己的核心壁垒，避免被 AI 取代？你觉得你在了解 ASR 和不了解 ASR 的情况下，利用 AI 完成这个项目会有什么不同？

2 基础任务

2.1 实验目标

本阶段实验目标是基于已有框架完成一个完整的拼音级自动语音识别系统。系统采用 Transformer Encoder + CTC 架构，将不定长语音序列转换为不定长拼音序列。根据实验要求，本阶段主要完成以下内容：

1. 完成训练代码，能够读取训练集和验证集，训练模型并保存验证集最优 checkpoint。
2. 完成推理代码，能够加载训练好的模型，对测试集语音生成拼音预测。
3. 完成评测代码，能够根据预测拼音和参考拼音计算 WER。
4. 记录实验操作、训练过程和测试结果，并简单说明对本项目的理解。

2.2 实验数据与环境

本实验使用标贝数据集中的 10000 条语音数据，识别目标为不带声调的拼音。数据集已经划分为训练集 8000 条、验证集 1000 条、测试集 1000 条。

表 1: 数据集划分

数据划分	样本数	文件
训练集	8000	dataset/split/train/wav.scp, dataset/split/train/pinyin
验证集	1000	dataset/split/dev/wav.scp, dataset/split/dev/pinyin
测试集	1000	dataset/split/test/wav.scp, dataset/split/test/pinyin

其中，wav.scp 的格式为：

```
1 语音id<TAB>语音路径
```

```
2
```

pinyin 的格式为：

```
1 语音id<TAB>空格分隔的拼音序列
```

```
2
```

例如：

```
1 000001 ka e er pu pei wai sun wan hua ti
```

```
2
```

实验在 Windows 环境下运行，主要依赖如下：

表 2: 实验环境

项目	配置
Python	3.12.7
PyTorch	2.8.0+cpu
torchaudio	2.8.0+cpu
运行设备	CPU
num_workers	0

2.3 实验操作与代码实现

2.3.1 数据读取与特征提取

数据读取由 `ASRDataset` 完成。它读取 wav.scp 和 pinyin，根据相同语音 id 建立音频路径和拼音标注之间的对应关系。每次取样时，先读取音频，再提取 Fbank 特征，最后把拼音序列转换为 id 序列。

音频特征使用 `torchaudio.compliance.kaldi.fbank` 提取，主要参数如下：

表 3: Fbank 特征参数

参数	取值
采样率	16000 Hz
Fbank 维度	80
帧长	25 ms
帧移	10 ms
窗函数	Hamming
预加重系数	0.97

提取后的特征形状为：

```
1 [帧数, 80]
```

由于不同语音的帧数不同，不同句子的拼音长度也不同，因此在构造 batch 时使用 `collate_fn` 进行 padding，并额外返回真实长度：

```
1 padded_features: [batch_size, max_frames, 80]
2 feature_lengths: [batch_size]
3 padded_labels: [batch_size, max_label_len]
4 label_lengths: [batch_size]
```

这些真实长度是 CTC Loss 必需的输入。

2.3.2 词表构建

词表由训练集拼音统计得到。训练集中共有 406 个不同拼音，加上 CTC 使用的 `<blank>`，最终词表大小为 407。CTC 的 blank id 固定为 0：

```
1 词表大小: 407
2 Blank ID: 0
```

训练时，标签序列只包含真实拼音 id；推理时，模型输出可能包含 blank 和重复 token，需要使用 CTC 解码规则还原为最终拼音序列。

2.3.3 模型结构

本实验基础模型为 Transformer Encoder + CTC，整体流程如下：

```
1 语音波形
2   -> Fbank 特征
3   -> 卷积下采样
4   -> 位置编码
5   -> Transformer Encoder
6   -> Linear + LogSoftmax
7   -> CTC Loss / CTC Greedy Decode
```

模型先通过两层二维卷积对时间维做约 4 倍下采样，以减少 Transformer 的序列长度和计算量。随后加入正弦位置编码，再送入 Transformer Encoder 建模上下文信息。最后通过线性层输出每一帧对应各个拼音 token 的概率分布。

本次训练使用的模型参数如下：

表 4: 基础模型参数

参数	取值
input_dim	80
d_model	128
nhead	4
num_encoder_layers	4
dim_feedforward	256
dropout	0.1
vocab_size	407

2.3.4 训练、推理与评测

训练入口为 `train.py`，执行命令为：

```
1 python train.py --epochs 10 --batch-size 8 --lr 1e-4 --grad-clip 5.0 --num-workers 0
2
```

训练过程中每个 epoch 后在验证集上计算 loss 和 WER。如果当前验证集 WER 优于历史最优结果，则保存为 `checkpoints/best.pt`，同时保存最近一次模型为 `checkpoints/latest.pt`。

推理入口为 `infer.py`，执行命令为：

```
1 python infer.py --checkpoint checkpoints\best.pt --output outputs\test_predictions.tsv
2
```

推理时加载验证集上最优的 checkpoint，恢复模型参数、模型配置和训练时保存的词表，对测试集逐 batch 前向计算，并使用 CTC 贪心解码规则生成拼音预测。

评测入口为 `evaluate.py`，执行命令为：

```
1 python evaluate.py --predictions outputs\test_predictions.tsv
2
```

本实验识别目标是拼音，因此 WER 以空格分隔的拼音 token 为单位计算：

```
1 WER = (S + D + I) / N
2
```

其中，S 表示替换错误，D 表示删除错误，I 表示插入错误，N 表示参考拼音序列中的 token 数。

2.4 实验结果

2.4.1 训练过程

基础模型共训练 10 个 epoch。训练 loss、验证 loss 和验证 WER 如表 5 所示。

表 5: 基础模型训练结果

Epoch	Train Loss	Dev Loss	Dev WER
1	5.9272	5.5152	100.0000%
2	4.6087	3.0083	64.5954%
3	2.5643	1.9301	48.6726%
4	1.8166	1.4602	39.8331%
5	1.4556	1.2357	35.7930%
6	1.2335	1.0937	32.7904%
7	1.0836	1.0106	31.3239%
8	0.9744	0.8997	28.4141%
9	0.8830	0.8630	27.4751%
10	0.8086	0.8046	25.8927%

训练曲线如下:

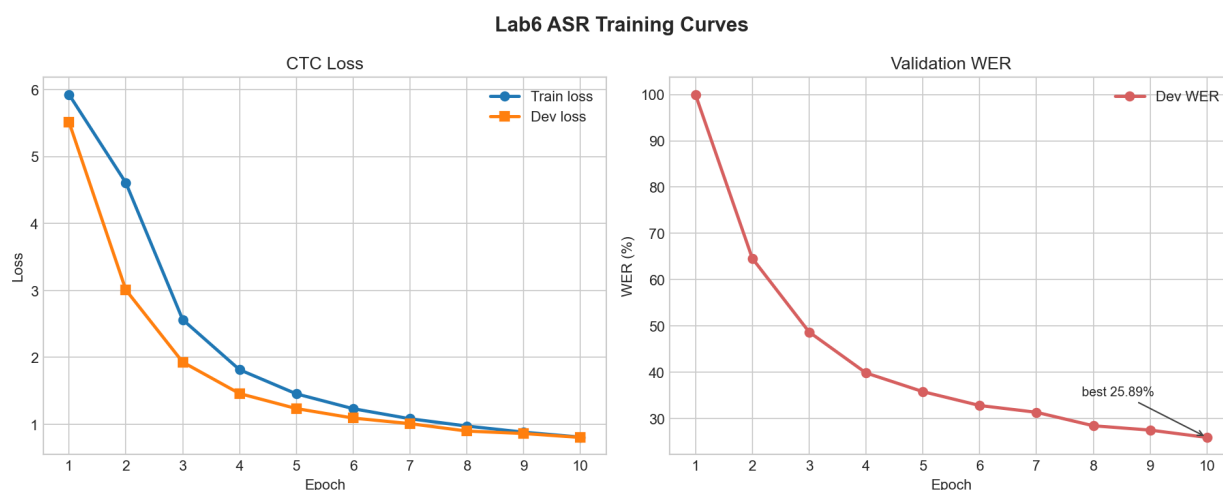


图 2.1: 基础模型训练曲线

从曲线可以看到, 训练 loss 和验证 loss 都稳定下降, 说明模型能够从训练数据中学习到语音和拼音之间的对应关系。验证集 WER 从第 1 个 epoch 的 100.0000% 下降到第 10 个 epoch 的 25.8927%, 下降趋势明显, 最终保存的最优模型来自第 10 个 epoch。

2.4.2 测试集结果

使用验证集上最优模型 `checkpoints/best.pt` 对测试集进行推理, 共生成 1000 条预测, 评测时没有缺失预测:

```

1 句子数: 1000
2 缺失预测: 0
3 测试集 WER: 24.2959%
4

```

最终测试集 WER 为 24.2959%, 低于 PPT 中提到的 30% 参考范围, 说明基础任务达到了预期效果。

2.4.3 预测样例分析

以下为测试集前 5 条样例的参考拼音和模型预测拼音：

表 6: 基础模型预测样例

ID	参考拼音	预测拼音
009001	wo men cheng shi de fu su you lai yu ta qiang you li de zheng ce	wo men cheng shi de fu su you lai yi kai qiang you li de zheng ce
009002	zhe yang zuo de yi ju jiu shi su ren tong e er san qi hao wen jian	zhe zu de yi ju yu jiu shi zu ren kou e er san xing tao wen ji
009003	zhe jiang hui hen da cheng du de huan jie yang lao jin de que kou	zhe jiang huo hui ke da cheng du huan jie yang lao jian de quan kao
009004	wei shen me ji feng wu gao xin jiu you na me da de po huai li	wei shen ma xing feng wu gao jiu you na me da de bo hua li
009005	gang mai de ku zi bu dao shi fen zhong ni jiu nong po le	gang lan de ku zi bu dao shi fen zhong ni jiu nong po le

从样例可以看出，模型已经能够识别出大部分句子结构和常见拼音，但仍存在一些替换错误和局部插入、删除错误。例如 `me/ma`、`po/bo`、`mai/lan` 等错误可能与拼音发音相近、上下文建模能力有限、模型规模较小有关。

2.5 折线图生成脚本

为了方便报告展示，我新增了脚本 `plot_training_curves.py`。该脚本会从 终端输出 `.txt` 中自动解析如下格式的训练日志：

```
1 epoch=10 train_loss=0.8086 dev_loss=0.8046 dev_wer=25.8927%
```

然后生成：

```
1 outputs/training_curves.png
2 outputs/training_metrics.csv
```

运行命令为：

```
1 python plot_training_curves.py --log 终端输出.txt --output outputs\training_curves.png --csv
   outputs\training_metrics.csv
```

其中 `training_curves.png` 用于报告展示，`training_metrics.csv` 便于后续复制到表格或重新绘图。

2.6 对本项目的理解

本实验让我更完整地理解了一个基础 ASR 系统的组成。语音识别不是简单的单输入单输出分类问题，而是一个典型的不定长序列到不定长序列的问题。输入语音经过分帧后得到较长的声学特征序列，而输出拼音序列通常短得多，两者之间没有人工标好的逐帧对齐关系。

在本项目中，Fbank 特征负责把原始波形转换成更适合模型学习的声学表示。卷积下采样用于减少时间步数量，降低 Transformer Encoder 的计算开销。Transformer Encoder 负责建模语音帧之间的上下文关系，使模型不仅根据当前帧判断拼音，还能利用前后语音信息进行判断。

CTC 是本项目中非常关键的一部分。它通过引入 blank 符号，允许模型在不知道帧级对齐的情况下学习语音和拼音序列之间的映射。训练时，CTC Loss 会把所有可能对齐路径的概率合并起来；推理

时，则可以使用较简单的贪心解码方法，先合并相邻重复 token，再删除 blank，得到最终拼音序列。

WER 则从应用角度衡量识别结果质量。相比只看 loss，WER 更接近最终任务目标，因为它直接反映预测序列和参考序列之间的替换、删除、插入错误。本次实验中，验证集 WER 随 epoch 从 100.0000% 下降到 25.8927%，测试集 WER 达到 24.2959%，说明模型确实学到了较有效的拼音识别能力。

同时，本项目也暴露出基础模型的局限：模型参数量较小、训练轮数有限，推理使用的是贪心 CTC 解码，没有使用语言模型或 beam search，因此在相近发音、长句局部结构上仍会出现错误。如果进一步改进，可以考虑更强的 encoder、SpecAugment 数据增强、beam search 解码、外部语言模型或预训练声学模型等方法。

3 进阶任务

3.1 进阶任务目标

基础任务中，我已经实现了 Transformer Encoder + CTC 的拼音级语音识别系统，并在测试集上得到 24.2959% 的 WER。进阶任务要求在基础系统之上继续和 AI 讨论并尝试改进，包括但不限于替换 encoder 框架、使用预训练 encoder 训练 CTC 等。

本阶段选择的优化方向是：将基础模型中的 Transformer Encoder 替换为 Conformer Encoder，并继续使用 CTC 进行训练和解码。选择该方向的原因是：Conformer 是语音识别任务中常用的 encoder 结构，它在 Transformer 自注意力的基础上加入卷积模块，能够同时建模长距离上下文和局部声学模式。相比普通 Transformer，它更贴合语音信号的时序特点。

3.2 进阶任务做了什么

3.2.1 保留基础系统，新增可选 Conformer 分支

本次改进没有删除或破坏原来的基础任务代码。原来的模型仍然可以通过默认参数运行：

```
1 python train.py --epochs 10 --batch-size 8 --lr 1e-4 --grad-clip 5.0 --num-workers 0
2
```

在此基础上，我新增了一个可选模型参数：

```
1 --model-type conformer
2
```

因此进阶模型可以通过下面的命令训练：

```
1 python train.py --model-type conformer --epochs 10 --batch-size 8 --lr 1e-4 --grad-clip 5.0
  --num-workers 0 --save-dir checkpoints_conformer
2
```

这样做的好处是：基础模型和进阶模型可以共用同一套数据处理、训练、推理、评测流程，只替换 encoder，方便公平对比。

3.2.2 新增 ASRConformerCTC 模型

在 `model/__init__.py` 中新增了 ASRConformerCTC。模型整体结构如下：

```

1 语音波形
2  -> Fbank 特征
3  -> 4 倍卷积下采样
4  -> 位置编码
5  -> Conformer Encoder
6  -> Linear 分类层
7  -> LogSoftmax
8  -> CTC Loss / CTC Greedy Decode
9

```

与基础模型相比，前端特征提取、卷积下采样、CTC 输出层、CTC Loss、贪心解码和 WER 评测都保持一致，主要变化集中在 encoder。

表 7: 基础模型与进阶模型结构对比

模块	基础任务	进阶任务
声学特征	80 维 Fbank	80 维 Fbank
下采样	2 层卷积，约 4 倍下采样	2 层卷积，约 4 倍下采样
Encoder	Transformer Encoder	Conformer Encoder
训练准则	CTC Loss	CTC Loss
解码方式	CTC Greedy Decode	CTC Greedy Decode
评测指标	拼音级 WER	拼音级 WER

3.2.3 修改模型构建与 checkpoint 恢复逻辑

在 `asr_utils.py` 中扩展了 `build_model`：当 `model_type="transformer"` 时构建原来的 ASRTransformerCTC；当 `model_type="conformer"` 时构建新增的 ASRConformerCTC。训练保存 checkpoint 时会记录模型类型和模型配置，因此推理和评测时不需要手动指定模型类型，脚本会自动根据 checkpoint 恢复正确模型。

例如，下面的命令可以直接加载 Conformer checkpoint：

```

1 python infer.py --checkpoint checkpoints_conformer\best.pt --output
   outputs\test_predictions_conformer.tsv
2 python evaluate.py --predictions outputs\test_predictions_conformer.tsv
3

```

3.3 进阶模型配置

Conformer 模型配置如表 8 所示。

表 8: Conformer 模型参数

参数	取值
model_type	conformer
input_dim	80
d_model	128
nhead	4
num_encoder_layers	4
dim_feedforward	256
dropout	0.1
conformer_kernel_size	31
vocab_size	407

训练超参数与基础模型保持一致: epochs=10、batch_size=8、lr=1e-4、grad_clip=5.0、num_workers=0, 优化器为 Adam, 损失函数为 CTCLoss。保持训练配置一致, 可以更直接地观察 encoder 替换带来的影响。

3.4 训练结果

Conformer 训练共进行 10 个 epoch, 训练 loss、验证 loss 和验证 WER 如表 9 所示。

表 9: Conformer 训练结果

Epoch	Train Loss	Dev Loss	Dev WER
1	5.4627	3.4734	67.0125%
2	2.4973	1.6369	37.9666%
3	1.3672	0.9809	22.9191%
4	0.8452	0.6339	15.0823%
5	0.5532	0.4536	10.7350%
6	0.3878	0.3325	8.0512%
7	0.2820	0.2829	7.5122%
8	0.2127	0.2258	6.2833%
9	0.1663	0.2103	5.8370%
10	0.1315	0.1803	5.1762%

训练曲线如下:

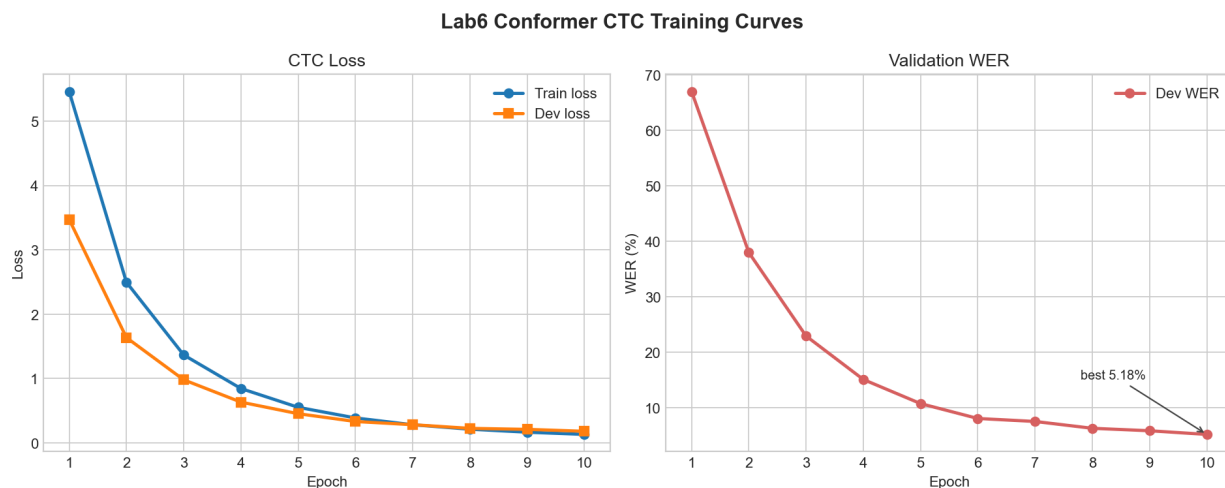


图 3.2: Conformer 训练曲线

从曲线可以看到，Conformer 的训练 loss、验证 loss 和验证 WER 都稳定下降。第 10 个 epoch 时，验证集 WER 达到 5.1762%。相比基础模型第 10 个 epoch 的验证集 WER 25.8927%，进阶模型有明显提升。

3.5 测试集结果

使用验证集上最优的 Conformer checkpoint:

```
1 checkpoints_conformer/best.pt
```

在测试集上生成 1000 条预测:

```
1 python infer.py --checkpoint checkpoints_conformer\best.pt --output
  outputs\test_predictions_conformer.tsv
```

然后进行评测:

```
1 python evaluate.py --predictions outputs\test_predictions_conformer.tsv
```

终端输出为:

```
1 评测文件: outputs\test_predictions_conformer.tsv
2 句子数: 1000, 缺失预测: 0, WER: 4.3773%
```

最终测试集 WER 为 4.3773%。

3.6 与基础模型的结果对比

基础模型与进阶模型的最终结果如下:

表 10: 基础模型与进阶模型结果对比

模型	Best Dev WER	Test WER	Final Train Loss	Final Dev Loss
Transformer Encoder + CTC	25.8927%	24.2959%	0.8086	0.8046
Conformer Encoder + CTC	5.1762%	4.3773%	0.1315	0.1803

对比图如下：

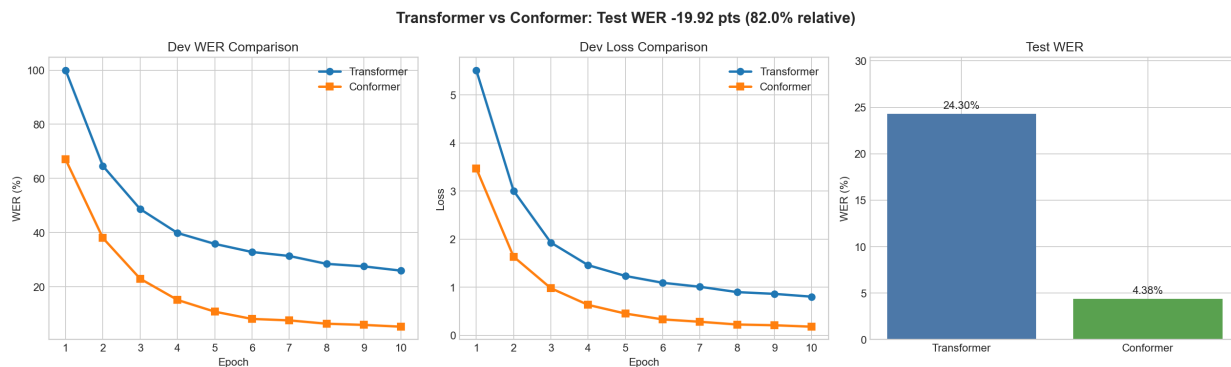


图 3.3: 基础模型与 Conformer 对比图

从结果可以看出：验证集 WER 从 25.8927% 降低到 5.1762%，绝对下降 20.7165 个百分点，相对下降约 80.0%；测试集 WER 从 24.2959% 降低到 4.3773%，绝对下降 19.9186 个百分点，相对下降约 82.0%。此外，Conformer 在第 3 个 epoch 的验证集 WER 已经达到 22.9191%，优于基础 Transformer 第 10 个 epoch 的 25.8927%，说明 Conformer 不仅最终效果更好，收敛速度也更快。

3.7 预测样例对比

以下展示测试集前 5 条样例。可以看到，Conformer 的预测整体更接近参考拼音，其中第 3 条完全正确。

表 11: Transformer 与 Conformer 预测样例对比

ID	参考拼音	Transformer 预测	Conformer 预测
009001	wo men cheng shi de fu su you lai yu ta	wo men cheng shi de fu su you lai yi kai	wo men cheng shi de fu su you lai yi ta
009002	qiang you li de zheng ce	qiang you li de zheng ce	qiang you li de zheng ce
009003	zhe yang zuo de yi ju jiu shi su ren tong	zhe zu de yi ju yu jiu shi zu ren kou e er	zhe yang zuo de yi ju jiu shi su ren tong
009004	e er san qi hao wen jian	san xing tao wen ji	e er san qi hou wen jian
009005	zhe jiang hui hen da cheng du de huan jie	zhe jiang huo hui ke da cheng du huan jie	zhe jiang hui hen da cheng du de huan jie
	yang lao jin de que kou	yang lao jian de quan kao	yang lao jin de que kou
009004	wei shen me ji feng wu gao xin jiu you na	wei shen ma xing feng wu gao jiu you na	wei sheng me ji feng wu gao xin jiu you
	me da de po huai li	me da de bo hua li	na le da de bo huai li
009005	gang mai de ku zi bu dao shi fen zhong	gang lan de ku zi bu dao shi fen zhong ni	gao lai de ku zi bu dao shi fen zhong ni
	ni jiu nong po le	jiu nong po le	jiu nong po le

样例中可以看到，Transformer 更容易出现连续多个 token 的替换和局部结构错误；Conformer 仍会犯少量近音替换错误，但多数句子的整体结构更完整，拼音序列更接近参考文本。

3.8 为什么 Conformer 有效果

3.8.1 Conformer 更适合语音信号

普通 Transformer Encoder 的核心是自注意力机制，擅长建模全局依赖，但语音信号有很强的局部连续性。一个音素或拼音的声学表现往往分布在相邻若干帧中，短时间内的局部变化非常关键。Conformer 在自注意力模块之外加入了卷积模块，使模型既能看到长距离上下文，又能捕捉局部声学模式。因此它比普通 Transformer 更适合作为语音识别 encoder。

3.8.2 卷积模块补足了局部建模能力

本项目使用 Fbank 特征，输入是按帧排列的声学序列。相邻帧之间通常存在明显的连续关系，例如声母到韵母的过渡、元音稳定段、短暂停顿等。Conformer 中的深度可分离卷积可以更直接地建模这些局部模式。基础 Transformer 也可以通过注意力学习局部关系，但它没有显式的局部归纳偏置；Conformer 的卷积模块相当于给模型加入“语音局部平滑和局部变化很重要”的先验，因此在同样训练轮数下更容易学到有效表示。

3.8.3 全局上下文和局部特征互补

语音识别不能只依赖局部发音。比如某些拼音发音相近，需要结合上下文判断；而长句中不同部分也存在语义和发音节奏上的关联。Conformer 保留了自注意力模块，所以仍然具备全局建模能力。因此 Conformer 的优势不是单纯加了卷积，而是把卷积的局部建模能力和 Transformer 的全局建模能力结合起来。这一点从实验结果也能体现：Conformer 不仅最终 WER 更低，而且从第 1 个 epoch 开始验证集 WER 就明显低于基础 Transformer。

3.8.4 相同训练配置下提升明显

本次进阶实验中，数据、Fbank 特征、词表、CTC Loss、解码方式、评测方式、训练 epoch、batch size、学习率等都与基础任务保持一致。主要差异是 encoder 从 Transformer 换成 Conformer。因此 WER 从 24.2959% 降到 4.3773%，可以较有说服力地说明：这次优化的主要效果来自 encoder 结构更适合 ASR 任务。

4 关于使用 AI 的一些问题

4.1 AI Coding 模式体验

本次实验我主要使用的是基于全项目上下文、支持多文件联动修改的 Vibe Coding 模式，而不是只让 AI 补单个函数的传统 AI Coding 模式。

原因是本实验不是一个孤立函数，而是一个完整 ASR 项目，涉及数据读取、特征提取、模型结构、训练、推理、评测和报告整理等多个环节。如果只逐个函数补全，很容易出现接口不一致、路径不统一、checkpoint 无法复用等问题。使用全项目上下文后，AI 可以同时理解 `dataprocess.py`、`model.py`、`train.py`、`infer.py`、`evaluate.py` 之间的关系，帮助我把训练、推理和评测流程串成闭环。

我的感受是，Vibe Coding 更像是在和 AI 一起做项目设计与实现，而不是简单让它写几行代码。它能明显提高开发效率，但前提是我需要知道整体目标，并持续检查 AI 给出的实现是否符合实验要求。

4.2 如何定位和修正 AI 代码问题

AI 生成的代码确实会出现不符合预期的情况。我主要通过运行、看报错、检查数据流和核对输出指标来定位问题。

例如在本实验中，我会让 AI 完成后先做小规模 smoke test，只跑 1 个 batch，检查数据能否读取、Fbank 特征形状是否正确、CTC Loss 是否能计算、checkpoint 是否能保存和加载。如果代码报错，我会根据报错位置回到对应模块检查输入输出形状、路径拼接、词表 blank id、padding 长度等关键点。

修正时，我不会直接相信 AI 的一次性生成结果，而是把问题拆开验证：先确认 Dataset，再确认模型 forward，再确认 loss，再确认推理和 WER。总而言之，通过具体报错和最小测试逐步定位，再修改相关代码基本就可解决；找到原因后再让 AI 根据具体问题修改，通常也能较好完成。

其实，我认为预防比补救更高效。使用 AI 时可以在正式写代码之前先让 AI 生成实现方案，人工检查是否有谬误或遗漏；再让 AI 明确会修改哪些文件、编写哪些模块和功能。这样能在代码初期预防一些问题，也能让我更清楚项目结构，后续出错时更容易溯源。

4.3 我和 AI 的长期关系与核心壁垒

我认为 AI 会长期成为我的编程助手、学习搭档和项目推进工具。它很适合帮助我快速搭建框架、解释陌生代码、生成实验脚本、整理报告和提出改进方向。但它不能完全替代我对任务本身的理解和判断。

为了避免被 AI 取代，我需要构建自己的核心壁垒：理解问题本质、掌握基础理论、能判断方案是否合理、能设计实验验证结果，并能在 AI 出错时定位原因。也就是说，AI 可以帮我写得更快，但我必须知道为什么这样写、哪里可能错、结果是否可信。

我很习惯在 AI 实现项目时提出一些猜想，而 AI 一般会对我的猜想做出一些可行性和有效性的评估。综合考虑后，如果方案可行，我再让 AI 执行这个想法。这就是我使用 Vibe Coding 的主要模式：我来猜想，AI 来评估，共同实现，逐步优化。

如果不了解 ASR，使用 AI 也许可以跑通代码，但很难判断 CTC、blank、WER、变长序列、特征长度这些细节是否正确，也难以解释结果为什么好或不好。了解 ASR 后，我能更清楚地给 AI 提需求，也能判断 Conformer 为什么比普通 Transformer 更适合语音识别。因此，AI 降低了完成项目的门槛，但专业理解决定了项目质量的上限。